

Data Structure

Level 1

Big $O(n)$: Linear Time
Function

Data Structure

Level 1

Complexity of an Algorithm

Big O

$O(n)$

The background of the slide features a complex, abstract network of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, the silhouettes of several people are visible, appearing to be in a collaborative setting, possibly a meeting or a workshop, with some individuals leaning over a table or screen.

Data Structure

Level 1

Quick Review

Big O for the following Algorithms:

Algorithm 1

```
char GetlastCharacter(string S1)
{
    return S1[ S1.length() - 1 ];
}
```

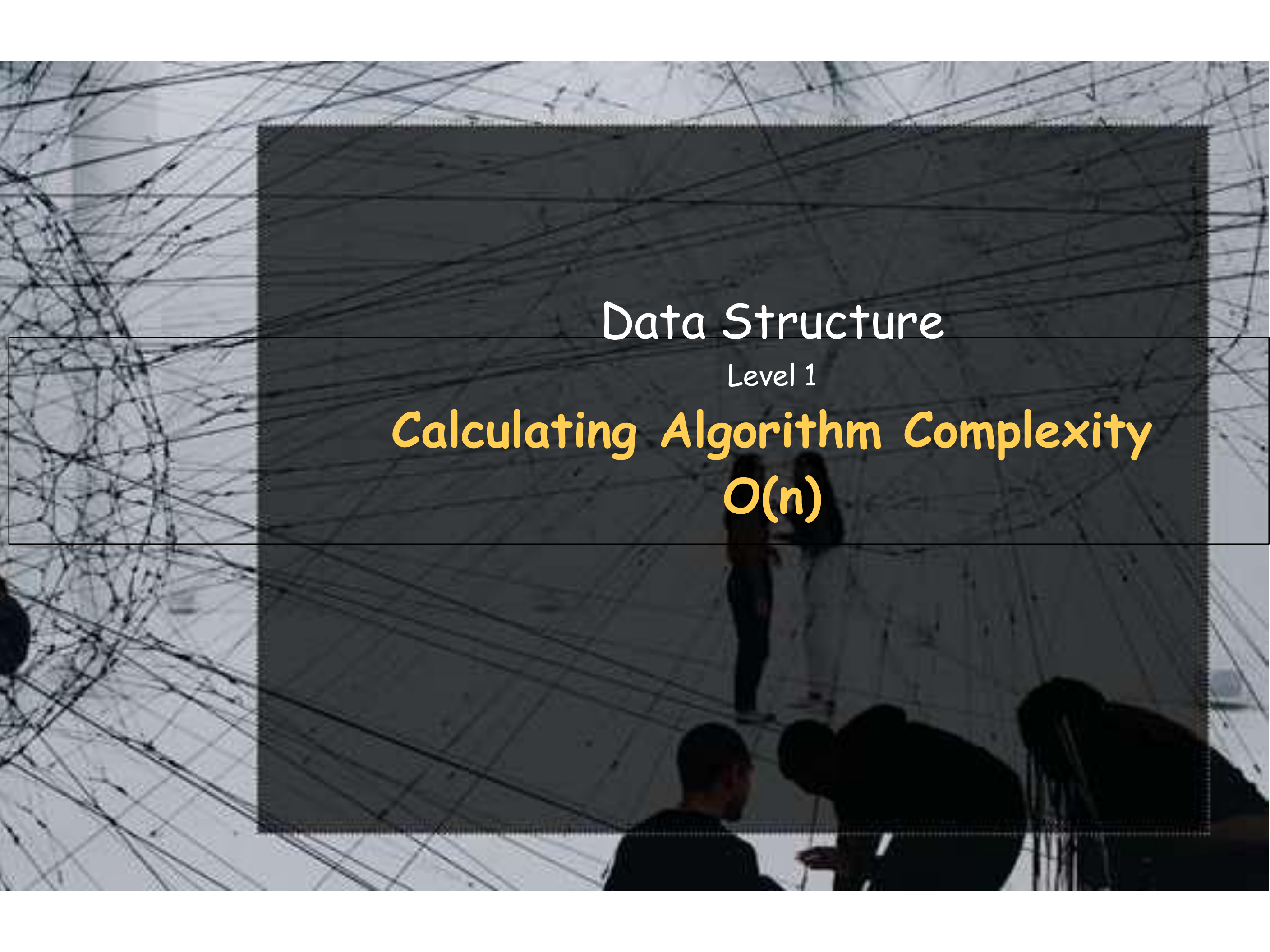
O(1)

Algorithm 2

```
char GetLastCharacter2(string S1)
{
    int n = S1.length() - 1 ;

    for ( int i = 0 ; i <= n ; i++ )
    {
        if ( i == n )
        {
            return S1 [ n ] ;
        }
    }
}
```

O(n)



Data Structure

Level 1

Calculating Algorithm Complexity $O(n)$

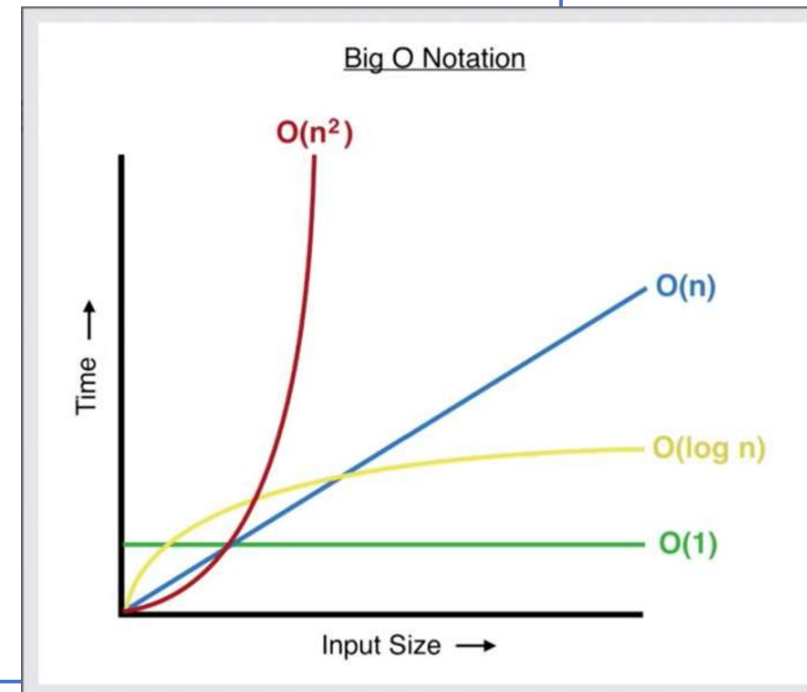
Calculating Algorithm Complexity:

```
char GetLastCharacter2(string S1)
{
    int n = S1.length() - 1;
    for ( int i = 0; i <= n; i++ )
    {
        if ( i == n )
        {
            return S1 [ n ] ;
        }
    }
}
```

Number of Steps outside loop = 4

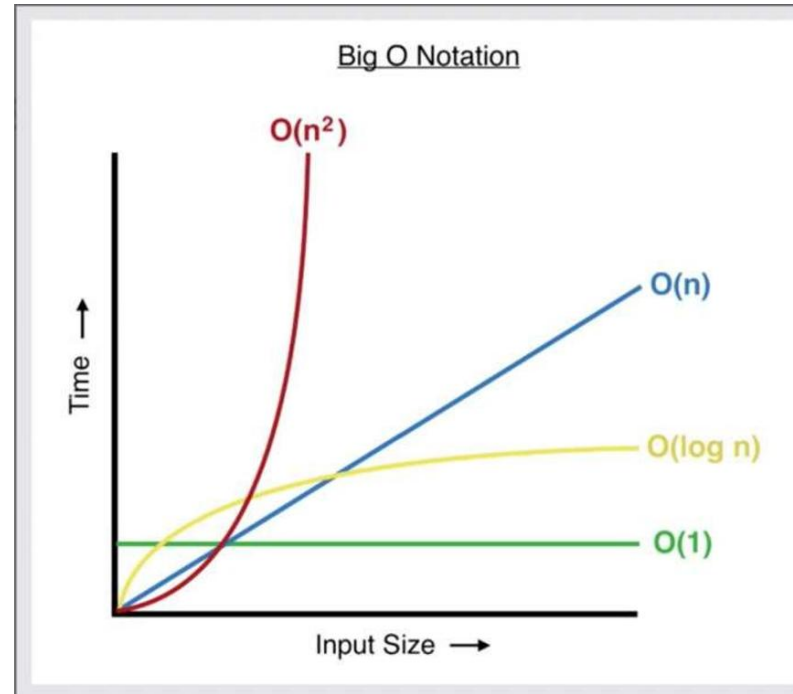
Number of Steps inside loop = 6

Big O = $6n + 4$
Remove Factors
 $O(n)$



Depends on array size, and relation is linear Time Function.

Comparing the two Algorithms



Faster

Slower

```
char GetLastCharacter(string S1)
{
    return S1[S1.length() - 1];
}
```

Big O = $4 * O(1) = 4 O(1)$
Remove Factor
 $O(1)$

```
char GetLastCharacter2(string S1)
{
    int n = S1.length() - 1;
    for (int i = 0; i <= n; i++)
    {
        if (i == n)
        {
            return S1[n];
        }
    }
}
```

Number of Steps outside loop = 4
Number of Steps inside loop = 6

Big O = $6n + 4$
Remove Factors
 $O(n)$

Thank you 😊